Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Kod kusurları ve refactoring eylemlerinin anlatım biçimi (00:07:17)
  - Her kod kusuru (code smell) için önce tanımı, nasıl ve neden ortaya çıktığı ve yol açtığı sorunlar anlatılır; ardından o kusuru giderebilecek refactoring eylemleri gösterilir.
  - İçerik Martin Fowler'ın Refactoring kitabından alınmıştır. Kitaptaki kod kusurlarının yaklaşık üçte biri derse dahil edilmiştir; ders bir giriş niteliğindedir.
  - Kod kusuru bir hata değildir: kullanılan programlama yaklaşımının önerdiği biçimin dışındaki kullanımlardır. Yazılımın esnekliğini düşürür, bağlaşımı yükseltir, uyumu düşürür.
- Duplicated Code (yinelenen kod) (ekrandan, 00:10:00)
  - Aynı kodun ya da çok benzer bir kod yapısının birden fazla yerde tekrarlanmasıdır. Genelde kopyala-yapıştır programlamanın sonucudur.
  - Kodun işlevselliği değişince, kodun tekrarlandığı her yerde aynı değişiklik yapılmalıdır. Sonuç bir bakım kabusudur.
  - Çözüm: Extract Method. Diğer çözümler: Pull Up Method, Form Template Method; kod ilişkisiz sınıflarda yineleniyorsa metot bir sınıfa (ya da yeni bir sınıfa) atanır, diğerleri onu kullanır (ekrandan, 00:30:00).
  - Slayttaki kod örneklerinde alt çizgiyle başlayan değişkenler (`_name`, `_orders`) nesnenin üye alanlarıdır; IDE'lerde bunu renk vurgusu gösterir.
- Extract Method (metot çıkartma) (ekrandan, 00:14:20 ve 00:21:30)
  - Bir metot içindeki kodun bir kısmını yeni bir metot olarak belirlemeye dayanır. Birçok kod kusurunu düzeltir (Duplicated Code, Long Method).
  - Yeni metoda amacını açıkça belirten, isabetli bir ad verilmelidir. Böylece alt düzey ayrıntılarla uğraşmadan kod anlaşılır ve yorum satırlarına gerek kalmaz.
  - Örnek 1: `printOwing(double amount)` içindeki ayrıntı yazdırma satırları `printDetails(double amount)` metoduna çıkarılır; `amount` yeni metoda parametre olarak geçirilir.
  - Örnek 2: `printOwing()` banner yazdırma, toplam borcu hesaplama ve ayrıntı yazdırma işlerini yapar. Bunlar `printBanner()`, `getOutstanding()` ve `printDetails(double outstanding)` metotlarına ayrılır; `getOutstanding()` içinde `while+Enumeration` yerine for-each döngüsü kullanılır.
- Long Method (aşırı uzun kod) (ekrandan, 00:20:00 ve 00:43:30)
  - Metot uzadıkça anlaşılması zorlaşır, değişikliklerden etkilenme olasılığı artar, birden fazla ilgi alanını kapsama olasılığı artar (uyumun düşmesi).
  - Çözüm: Extract Method (uyumlu, belli bir ortak amaca yönelik komutlar seçilerek bir metot altında toplanır). Diğer çözümler: Decompose Conditional, Replace Temp with Query, Replace Method with Method Object.
- Pull Up Method (metodu yukarı çekme) (ekrandan, 00:34:40 ve 00:40:00)
  - Kardeş sınıflarda yinelenen metotlar üst sınıfa çekilerek bir tek yerde kodlanır. Örnek: `Salesman` ve `Engineer` sınıflarındaki `getName` metodu `Employee` üst sınıfına taşınır.
  - Pull Up / Push Down adlarındaki yukarı-aşağı, kalıtım hiyerarşisinde üst ya da alt sınıfa doğru hareketi anlatır. Çıkarılan metot aynı sınıfta kalırsa Extract Method, üst sınıfa konursa Pull Up Method'dur.
  - Özel durum: `RegularCustomer` ve `PreferredCustomer` sınıflarında `createBill(Date)`

ve `chargeFor(start: Date, end: Date)` vardır, `chargeFor` iki sınıfta çok farklı kodlanmıştır. Sonraki halde `createBill` üst sınıf `Customer`'a taşınır, `chargeFor` ise `Customer`'da soyut tanımlanır ve alt sınıflarda kalır. Kod daha da benzerse Template Method tasarım kalıbı kullanılabilir.

- Decompose Conditional (karar verme komutunu parçalama) (ekrandan, 00:50:00)
  - Aşırı karmaşık if komutlarını sadeleştirmek anlaşılabilirliği artırır. Karar verme (if, switch-case) ve döngülerin yoğun olması çoğu kez kodu uzatır.
  - Örnek: `date.before(SUMMER_START) || date.after(SUMMER_END)` koşulu `notSummer(date)` metoduna, iki daldaki hesaplamalar `winterCharge(quantity)` ve `summerCharge(quantity)` metotlarına çıkarılır. Yardımcı metotlar `private` tanımlanır.
- Long Parameter List (uzun parametre listesi) (ekrandan, 01:12:00)
  - Bir metodun parametre sayısının fazla olması, hatta bu parametrelerden bazılarının başka metotlarda da birlikte geçmesidir. Bu parametreler birlikte bir varlığın durum bilgisinin bir kısmını (1) ya da tamamını (2) oluşturur.
  - Çözüm: (1) Introduce Parameter Object, (2) Preserve Whole Object.
- Introduce Parameter Object (parametre nesnesi oluşturma) (ekrandan, 01:20:00)
  - Uygun parametreler üye alan olarak yeni bir sınıfta toplanır ve o türden bir nesne parametre olarak kullanılır.
  - Örnek: `Customer` sınıfında `amountInvoicedIn`, `amountReceivedIn`, `amountOverdueIn` metotlarının hepsi `(start: Date, end: Date)` alır; sonraki halde hepsi `(DateRange)` alır.
  - Yalnız parametre listesini kısaltmak için yapılmaz: `DateRange` yalnız iki veriyi birleştiren bir kayıt yapısı olarak kalmaz, `isWithinRange(Date)` gibi uygun metotları da içerir; böylece Decompose Conditional ya da Extract Method düzenlemesi de yapılmış olunur.
- Preserve Whole Object (tüm nesneyi gönder) (ekrandan, 01:16:40)
  - Bir nesnenin çeşitli üyelerini aynı metodun parametreleri olarak göndermek yerine nesnenin kendisi gönderilir.
  - Örnek: `daysTempRange().getLow()` ve `daysTempRange().getHigh()` ile `low`, `high` geçici değişkenleri alınıp `plan.withinRange(low, high)` çağrılıyordu; sonraki halde `plan.withinRange(daysTempRange())` çağrılır ve `withinRange` buna göre değiştirilir. Geçici değişkenler de ortadan kalkar.
  - Zayıf yönler (slayt): (1) bağımlılık artabilir, parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi sınıf arasında bağımlılık oluşur; (2) metot imzası değiştiği için geriye doğru uyumluluk kaybedilir, refactoring tanımı esnetilmiş olur. Bunlardan biri bile çok sakıncalıysa düzenleme kullanılmaz.
- Replace Temp with Query (geçici değişken yerine erişim metodu kullanımı) (ekrandan, 01:30:00, 01:32:40, 01:40:00)
  - Bir ifadeyi geçici değişkende saklayarak kullanmak yerine ifadeden metot(lar) çıkarılır.
  - Örnek: `getPrice()` içindeki `basePrice` ve `discountFactor` geçici değişkenleri `basePrice()` ve `discountFactor()` sorgu metotlarına dönüşür; `getPrice()` tek satır olur: `return basePrice() * discountFactor();`.
  - Geçici değişkenin değeri bir kez atanıp sonra değişmiyorsa onu ortadan kaldırmak kolaydır; birkaç yerde değişiyorsa her biri için ayrı metot çıkarmak gerekebilir.
  - Avantaj: kopyala-yapıştır kodlamayı daha ortaya çıkmadan azaltır; hata ya da değişiklik yalnız sorgu metodunda düzeltilir.
  - Tartışma: başarımı düşürür (fazladan metot çağırma yükü, aynı işlemin tekrarı). Şerh: modern derleyiciler bu ek yükü ortadan kaldırmış ya da önemsizleştirmiştir.
  - İkinci örnek: `printRecipt()` döngüsündeki `thisAmount` geçici değişkeni kaldırılır, `item.getCharge()` iki yerde çağrılır; birikimli toplam için `totalAmount` kalmak zorundadır.
- Replace Method with Method Object (metot yerine yeni türden nesne kullanımı) (ekrandan, 01:43:50)
  - Metot öyle uzun ve geçici yerel değişkenleri öyle karışık kullanıyor ki Extract Method yapılamıyorsa, yeni bir sınıf oluşturulur, bütün metot o sınıfa taşınır, yerel değişkenler o sınıfın üyeleri yapılır. Ardından bilinen düzenleme eylemleri daha rahat yürütülür.
  - Yeni sınıfın adı taşınan metodun adına benzer verilir. Örnek: `Order.price()` içindeki

`primaryBasePrice`, `secondaryBasePrice`, `tertiaryBasePrice` ve uzun hesaplama `PriceCalculator` sınıfına taşınır; `price()` yalnız `return new PriceCalculator(this).compute()` yapar.

Hocanın Özellikle Vurguladığı Kısımlar

- Kod kusurlarının ve refactoring eylemlerinin İngilizce adları
  - Hoca bu kusurların ve eylemlerin literatürdeki İngilizce adlarıyla bilinmesini istiyor; İngilizce ad slayt başlığında, uygun Türkçe karşılık altında yer alıyor.
- Anlamlı metot adları yorum satırlarının yerini alır
  - Extract Method sonrasında metot adları iyi seçildiği için yorum satırlarına gerek kalmadı. Hedef, hiç yorum yazmadan kodu okuyanın anlayabilmesidir.
- Metot sayısının artması bir dezavantajdır ama kazanımların yanında küçük kalır
  - Parçalar küçük "lokmalar" haline gelir; her biri hızlıca okunur, anlaşılır, düzeltilir. Çıkarılan metotlar başka yerlerde de kullanılabilir ve yinelenen kod daha ortaya çıkmadan engellenir.
- Metot çağrısının ek yükü (context switching overhead) çoğunlukla eskide kaldı
  - Modern derleyiciler ve yorumlayıcılar (JIT) bu yükü önemli oranda azaltmıştır; yok denecek kadar azdır.
- Geçici değişkenler bir zaman-bellek ödünleşimidir
  - Hesap tekrarını engeller ama bellek ihtiyacını artırır ve metodu uzatmaya meyleder. Hoca mümkünse geçici değişken kullanılmamasını, ama birikimli toplam gibi zorunlu durumlarda kullanılmasını söylüyor.
- Başarım kaygısını ölçerek karara bağlamak
  - Refactoring'den önce ve sonra profiler ile başarım ölçülür; fark kabul edilen sınırı aşarsa eylemler geri alınır ya da sürüm denetim sisteminde önceki sürüme dönülür (rollback).
  - Darboğaz genelde refactoring yapılan yerlerin dışındadır; birkaç yüz milisaniyelik ek yük, saniyeler süren bir darboğazın yanında göz ardı edilebilir.
  - Ancak hesaplama çok masraflıysa (başka sunucuda, ağ üzerinden yapılan hesaplar gibi) Replace Temp with Query yapılmamalıdır.
- Refactoring eylemlerinin de dezavantajları vardır
  - Preserve Whole Object / Introduce Parameter Object bağlaşımı artırabilir ve geriye doğru uyumluluğu bozabilir; Replace Method with Method Object yeni bir sınıf ve yeni ilişki ekler. Kazanım ve kayıp tartılarak karar verilir.
- UML gösterim titizliği
  - Hoca Fowler'ın şemalarındaki eksikleri belirtti: görünürlük ve dönüş türü (`+getName() : String`) gösterilmemiş; soyut sınıf ve metot yalnız italikle gösterilmiş, `<<abstract>>` / `{abstract}` gibi açık işaret daha iyi olurdu; bazı kutularda metotlar ortada, üye alanlar altta gösterilmiş. Hoca önce üye alanların, sonra metotların gösterilmesini istiyor; sınavda bunlara dikkat edilmeli.

Kısa Tekrar Notları

- Kod kusuru hata değildir; bağlaşımı yükseltir, uyumu düşürür.
- Duplicated Code → Extract Method, Pull Up Method, Form Template Method, metodu bir (yeni) sınıfa atama.
- Long Method → Extract Method, Decompose Conditional, Replace Temp with Query, Replace Method with Method Object.
- Long Parameter List → Introduce Parameter Object (durum bilgisinin bir kısmı), Preserve Whole Object (tamamı).
- Extract Method: kodun bir kısmı yeni, isabetli adlı bir metoda çıkarılır; yorum gereksizleşir.
- Pull Up Method: kardeş sınıflardaki aynı metot üst sınıfa çekilir; çok farklı kodlanmışsa üst sınıfta soyut tanımlanır.
- Decompose Conditional: karmaşık koşul ve dallar `notSummer`, `winterCharge`, `summerCharge` gibi metotlara ayrılır.
- Introduce Parameter Object: `(start, end)` → `DateRange`; sınıfa davranış (`isWithinRange`) da eklenir.

- Preserve Whole Object: `withinRange(low, high)` → `withinRange(daysTempRange());` bağımlılık ve imza değişikliği riski.
- Replace Temp with Query: `basePrice`, `discountFactor` geçici değişkenleri sorgu metotlarına dönüşür; başarım kaygısı profiler ile ölçülür.
- Replace Method with Method Object: uzun metot `PriceCalculator` gibi yeni bir sınıfa taşınır, yerel değişkenler üye alan olur.

Detaylı Açıklamalar

- Extract Method örnekleri (ekrandan, 00:14:20 ve 00:21:30)

```
/*** ÖNCE: ***/
void printOwing() {
  Enumeration e = _orders.elements();
  double outstanding = 0.0;

  // print banner
  System.out.println("*****************");
  System.out.println("* Customer Owes *");
  System.out.println("*****************");

  // calculate outstanding
  while (e.hasMoreElements()) {
    Order each = (Order) e.nextElement();
    outstanding += each.getAmount();
  }

  //print details
  System.out.println ("name:" + _name);
  System.out.println ("amount" + outstanding);
}

/*** SONRA: ***/
void printOwing() {
  printBanner();
  printDetails(getOutstanding());
}
void printBanner() {
  System.out.println("*****************");
  System.out.println("* Customer Owes *");
  System.out.println("*****************");
}
double getOutstanding() {
  double outstanding = 0.0;
  for (Order each : _orders)
    outstanding += each.getAmount();
  /* for-each ile döngü kısaldı! */
  return outstanding;
}
void printDetails (double outstanding) {
  System.out.println ("name:" + _name);
  System.out.println ("amount" + outstanding);
}
```

  - Kod parçadır, derlenmedi. Önceki halde metodun üç ayrı iş yaptığı yorum satırlarından da görülür. Sonraki halde `outstanding` geçici değişkeni yalnız `getOutstanding()` içinde gerekir; `printOwing()` bir bakışta anlaşılır, hata ya da değişiklik ilgili alt metotta aranır.
  - İlk örnekte (slayt 14) metot üç komutluktu ve bölünce toplamda biraz uzadı. Hoca bunun yalnız eylemin nasıl yapıldığını göstermek için olduğunu, kazancın uzun metotlarda belirginleştiğini söyledi.

- Pull Up Method ve soyut metot (ekrandan, 00:40:00)
  - Önce: `Customer` (`lastBillDate`, `addBill(dat: Date, amount: double)`), alt sınıflar `RegularCustomer` ve `PreferredCustomer` (her ikisinde `createBill(Date)` ve `chargeFor(start: Date, end: Date)`; notta "çok farklı kodlanmışlar").
  - Sonra: `Customer` (`lastBillDate`; `addBill`, `createBill(Date)`, soyut `chargeFor(start: Date, end: Date)`), alt sınıflarda yalnız `chargeFor` kalır. Not: "chargeFor metodu üst sınıfta soyut tanımlandı".
  - Hocaya göre ortak kod pek olmasa da her iki müşteri türü `Customer` gibi erişilip faturalandırılabilir; üst sınıftaki soyut imza, alt sınıflarda gövdenin mutlaka tanımlanması gerektiğini de bildirir.
  - Form Template Method'da değişmeyen kısımlar üst sınıftaki şablon metotta, değişen kısımlar alt sınıflarda kodlanır; şablon metot alt sınıfta ezilmemesi için `final` tanımlanmalıdır.
- Decompose Conditional (ekrandan, 00:50:00)

```
/*** ÖNCE: ***/
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;

/*** SONRA: ***/
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);
private boolean notSummer(Date date) {
    return date.before (SUMMER_START) ||
    date.after(SUMMER_END);
}
private double summerCharge(int quantity) {
    return quantity * _summerRate;
}
private double winterCharge(int quantity) {
    return quantity * _winterRate + _winterServiceCharge;
}
```

  - Kod parçadır, derlenmedi. Hoca koşulu bir zaman çizgisi üzerinde çözümledi: yaz başlangıcından önceki ve yaz sonundan sonraki tarihler, yani yaz olmayan dönemler kış ücretine girer. İlkbahar ve sonbaharın da kış ücretine girmesi iş kuralı kabul edildi.
  - Koşul tamamen ortadan kalkmadı, ama ayrı ve odaklanmış bir metodun içine alındı; üstteki kod "yaz değilse kış ücreti, değilse yaz ücreti" diye okunur.
  - Hoca, koşulu "tarih yaz başlangıcından sonra ve yaz sonundan önceyse `false`, değilse `true` döndür" biçiminde yazmanın kendisine daha anlaşılır geldiğini söyledi.

  Ders sonrası not: Hocanın önerdiği `after(SUMMER_START) && before(SUMMER_END)` biçimi sınır günlerde slayttaki koşulla aynı sonucu vermez. Tarih tam `SUMMER_START` ya da `SUMMER_END` ise slayttaki `notSummer` `false` (yaz) döndürür, önerilen biçim ise `true` (yaz değil) döndürür. Eşdeğer yazım `return !(!date.before(SUMMER_START) && !date.after(SUMMER_END));` olur (tamsayı tarihlerle python ile denetlendi).
- Introduce Parameter Object ve Preserve Whole Object (ekrandan, 01:16:40 ve 01:20:00)
  - `DateRange` sınıfı eklenince `Customer` ile `DateRange` arasında en azından bir kullanma ilişkisi doğar. Hoca kullanma ilişkisini sahiplik kadar ağır saymıyor ("yarım ağırlık"), ama bağlaşım az da olsa artar.
  - Refactoring'in katı tanımı "dış dünyaya açılan arayüzü değiştirmeden iç yapıyı değiştirmek" ise, public bir metodun imzasını değiştiren bu eylemler geriye doğru uyumluluğu bozar. Geriye doğru uyumluluk ya da düşük bağlaşım özellikle önemliyse uzun parametre listesi olduğu gibi bırakılabilir.
  - Hocaya göre Preserve Whole Object "bir taşla iki kuş vurur": parametre listesini toparlar ve `low`, `high` gibi geçici değişkenleri de ortadan kaldırır. Ayrıca `withinRange`

ileride nesnenin başka üyelerine de ihtiyaç duyarsa parametre listesinin yeniden uzaması önlenmiş olur.

Ders sonrası not: Hoca bağımlılık ve geriye doğru uyumluluk zayıf yönlerini anlatırken eylemin adını "Introduce Parameter Object" diye andı (01:20:48, 01:23:51). Ekrandaki slaytta bu iki zayıf yön Preserve Whole Object başlığı altında yazılıdır (ekrandan, 01:16:40). Her iki eylem de metot imzasını değiştirdiği ve yeni bir sınıfa bağımlılık getirdiği için, büyük olasılıkla, söylenenler ikisi için de geçerlidir.

- Replace Temp with Query (ekrandan, 01:30:00 ve 01:40:00)

```
/*** ÖNCE: ***/
double getPrice() {
    int basePrice = _quantity * _itemPrice;
    double discountFactor;
    if (basePrice > 1000) discountFactor = 0.95;
    else discountFactor = 0.98;
    return basePrice * discountFactor;
}

/*** SONRA: ***/
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
    if (basePrice() > 1000) return 0.95;
    else return 0.98;
}
```

  - Kod parçadır, derlenmedi. Önceki halde 2 çarpma yapılır. Sonraki halde `basePrice()` hem `getPrice()` hem `discountFactor()` içinde çağrıldığı için çarpma sayısı 3'e çıkar. Hocaya göre saniyede milyonlarca işlem yapan bir bilgisayarda bu fark hissedilmez.
  - `printRecipt()` örneğinde `thisAmount` kaldırılınca `item.getCharge()` iki kez çağrılır. Hocanın hesabı: `getCharge` 1 ms sürerse 100 kayıtta 100 ms yerine 200 ms harcanır; fazladan 100 ms hissedilmez.
  - Hoca darboğazı köprüye girerken beş şeridin üç şeride düştüğü trafik örneğiyle anlattı: akışı belirleyen o daralan noktadır, iyileştirme de oraya yapılmalıdır.

- Replace Method with Method Object (ekrandan, 01:43:50)

  - Önce: `class Order { double price() { double primaryBasePrice; double secondaryBasePrice; double tertiaryBasePrice; // long computation; ... } }`.
  - Sonra: `Order` (metot: `price()`) ile `PriceCalculator` (üye alanlar: `primaryBasePrice`, `secondaryBasePrice`, `tertiaryBasePrice`; metot: `compute()`). `Order`'dan `PriceCalculator`'a kesikli bağımlılık oku, `PriceCalculator`'dan `Order`'a 1 çokluklu ilişki vardır. `price()` notu: `return new PriceCalculator(this).compute()`.
  - `Order`'ın private üye alanlarını tek tek göndermek uzun parametre listesi doğurur; bu yüzden `Order` kendini bütün olarak (`this`, Preserve Whole Object) yapıcıya ya da `compute` metoduna verir.
  - Dezavantaj: sınıf sayısı bir artar, bağlaşım da artar. `PriceCalculator` iyice refactor edildikten sonra metot yeniden `Order`'a alınabilir ve bağlaşım artışı geri alınır.

Ders sonrası not: Hoca metot çağrısında çağıran metodun durum bilgisinin "heap'te" bir yığına atıldığını söyledi (00:27:28). Yerel değişkenler ve dönüş adresi çağrı yığınında (stack) tutulur; heap nesneler içindir. Ayrıca hocanın andığı out-of-order execution, büyük olasılıkla, derleyicinin değil işlemcinin yaptığı bir iyileştirmedir; metot çağrısının yükünü derleyici ve JIT tarafında asıl azaltan, büyük olasılıkla, metodun çağrıldığı yere gömülmesidir (inlining).

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.